

Project Specification: Multi-Tenant WhatsApp AI Shopping Assistant

Prepared by: Saad Hanif Taj

1. Core Requirements & Features

A custom, scalable WhatsApp conversational commerce platform allowing retail stores to sell directly to customers via AI, managed through a centralized web dashboard.

WhatsApp AI Assistant

- **Conversational Onboarding:** Welcomes customers and uses natural language to understand their needs.
- **Dynamic Catalog Browsing:** Displays categories, rich media (images, prices, descriptions) via native WhatsApp UI carousels.
- **Smart Recommendations:** Uses AI to suggest products based on customer preferences and live inventory.
- **Automated Support (Q&A):** Answers questions regarding sizing, colors, availability, and specific store policies.
- **In-Chat Cart & Checkout:** Allows users to add items to a cart, collecting delivery/collection details natively in the chat.
- **Human Handover:** Seamlessly transfers the conversation to a human staff member upon request or low AI confidence.

Merchant & Multi-Tenancy (Admin Dashboard)

- **Multi-Store Support:** Isolated environments for different retail stores (separate catalogs, branding, and orders).
- **No-Code Catalog Management:** Web dashboard for store owners to manually upload products, set prices, and update stock (with CSV bulk upload support).
- **Order Management:** Visual Kanban board (New, Processing, Completed) to track and fulfill orders.
- **Instant Notifications:** Sends completed order alerts to the store owner via Email and WhatsApp.
- **Scalability:** Architecture built to easily support future phases (Stripe/payment integration, delivery tracking URLs, loyalty broadcasts).

2. Proposed System Workflow

A. The Customer Journey (WhatsApp)

1. **Initiation:** Customer texts the store's WhatsApp number.
2. **Intent Matching:** AI determines if the user is asking a question, browsing, or looking for a specific item.
3. **Product Discovery:** AI triggers a database search and returns a visually appealing WhatsApp Interactive Carousel of matching, in-stock products.
4. **Cart & Checkout:** Customer clicks "Add to Cart" on items. Once ready, the AI prompts for delivery details and confirms the order.
5. **Completion:** Customer receives an order summary and the AI notifies the store owner.

B. The Merchant Journey (Web Dashboard)

1. **Setup:** Store owner logs into the web dashboard and uploads their product inventory (images, descriptions, prices).
2. **Monitoring:** Owner receives a WhatsApp ping: *"New order received from [Customer Name]"*.
3. **Fulfillment:** Owner checks the dashboard, views the exact order details, and moves the order status from "Pending" to "Out for Delivery."

3. Recommended Tech Stack (Lean & Serverless)

This stack avoids heavy DevOps, utilizing Vercel for fast, scalable deployment and managed services to keep monthly running costs minimal.

- **Frontend & Backend (Dashboard & API):** Next.js (React) deployed on **Vercel**. Vercel Serverless Functions will serve the admin dashboard and handle incoming Meta webhooks.
- **Database & Vector Search:** Supabase (Managed PostgreSQL). Provides multi-tenant data isolation via `store_id`, secure user authentication for the dashboard, and pgvector for AI semantic product search.
- **Message Queue (Crucial for Webhooks):** Upstash Redis. Serverless message queue to handle sudden spikes in WhatsApp messages so Vercel functions process the AI requests in the background without timing out Meta's webhook limits.
- **AI Engine:** OpenAI API (gpt-4o-mini) utilizing native Tool Calling (Function Calling). Strict JSON tools (e.g., `search_catalog`, `add_to_cart`) ensure the AI only quotes real, in-stock items.
- **Messaging Gateway:** Official Meta WhatsApp Cloud API. Direct REST API connection to avoid third-party aggregator markup fees.